

ABSTRACT

Students preparing for examinations work with large volumes of study material such as lecture notes, textbook chapters and reference articles. Keyword search matches characters rather than meaning, so it fails whenever a question is phrased differently from the text. General-purpose chat assistants answer fluently but are not grounded in the student's own syllabus material, cannot cite a source, and may fabricate content. Cloud notebook services provide grounding but require an account, constant connectivity and the upload of personal material to a service the student does not control.

This project presents StudyLens, a web-based study assistant built on Retrieval-Augmented Generation (RAG). A student creates a notebook and uploads PDF, DOCX, TXT or Markdown files, or adds a web page URL. The application extracts the text, splits it into overlapping chunks of 256 words, converts each chunk into an embedding vector and stores the vectors in a local ChromaDB collection dedicated to that notebook. When a question is asked, it is embedded with the same model, the three most similar chunks are retrieved by cosine similarity, and those chunks are placed in the prompt sent to the language model. The answer is streamed back token by token over Server-Sent Events and is displayed with the name of the source document it came from.

The same index is reused to produce three further study aids without re-reading the original file: a multiple-choice quiz with automatic validation and scoring, an interactive concept mind map, and a slide deck with four layout types and speaker notes that exports as a single self-contained HTML file. The backend uses FastAPI and SQLAlchemy over SQLite; the frontend uses React 18, TypeScript, Vite and Tailwind CSS.

The system was tested with a 68-chunk web source and a PDF source. Indexing completed in 15.25 seconds and the automatic summary in a further 4.04 seconds. Grounded answers began streaming after 2.42 seconds and completed in 3.38 seconds on average. A five-question quiz took 4.01 seconds, a fifteen-node mind map 8.71 seconds and a ten-slide deck 15.94 seconds. All fourteen functional test cases and three automated unit tests passed. Two defects were found and corrected: a streaming-

response fault that blocked all four generation features, and an option-parsing fault that corrupted quiz answer choices.

Keywords: StudyLens, Retrieval-Augmented Generation, Large Language Model, Vector Database, ChromaDB, Semantic Search, Text Embedding, Document Chunking, Server-Sent Events, FastAPI, React, Quiz Generation, Mind Map, Slide Generation, Exam Preparation, Grounded Answering, Source Citation.

INDEX

S. No.	CONTENT	PAGE NUMBER
	ACKNOWLEDGEMENT	i
	DECLARATION	ii
	ABSTRACT	iii
	LIST OF FIGURES	vi
	LIST OF TABLES	vii
	LIST OF ABBREVIATIONS	viii
1	INTRODUCTION	1 – 3
2	LITERATURE REVIEW	4 – 6
3	METHODOLOGY	7 – 13
4	IMPLEMENTATION	14 – 17
5	TESTING & DEPLOYMENT	18 – 21
6	RESULTS AND DISCUSSION	22 – 29
7	CONCLUSION AND FUTURE SCOPE	30 – 32
	REFERENCES	33 – 34

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 2.1	Limitations of Existing Approaches and the Research Gap	6
Figure 3.1	System Architecture of StudyLens	8
Figure 3.2	Document Ingestion and Indexing Pipeline	9
Figure 3.3	Retrieval-Augmented Query Workflow	10
Figure 3.4	Use Case Diagram of StudyLens	12
Figure 3.5	Level-1 Data Flow Diagram	12
Figure 3.6	Sequence Diagram of a Chat Request	13
Figure 4.1	Database Schema and Vector Store Relationship	15
Figure 6.1	Notebook Dashboard	22
Figure 6.2	Notebook Workspace with Indexed Sources	22
Figure 6.3	Automatically Generated Document Summary	23
Figure 6.4	Streaming Answer with Source Citation	23
Figure 6.5	Completed Grounded Answer	24
Figure 6.6	Generated Multiple-Choice Quiz	24
Figure 6.7	Quiz Answer Validation and Score	25
Figure 6.8	Mind Map Generation Progress	25
Figure 6.9	Generated Concept Mind Map	26
Figure 6.10	Slide Deck — Title Layout	26
Figure 6.11	Slide Deck — Bullets Layout	27
Figure 6.12	Exported Offline Deck — Title Slide	27
Figure 6.13	Exported Offline Deck — Content Slide	27

LIST OF TABLES

Table No.	Table Title	Page No.
Table 3.1	Technologies Used in the System	8
Table 3.2	Major System Modules	9
Table 3.3	Configurable Retrieval Parameters	10
Table 4.1	Development and Testing Environment	14
Table 4.2	Relational Database Schema	15
Table 4.3	Frontend Components	17
Table 4.4	REST and Streaming API Endpoints	17
Table 5.1	Automated Unit Test Results	18
Table 5.2	Functional Test Cases	19
Table 5.3	Defects Identified and Resolved	20
Table 5.4	Measured Response Times	21
Table 6.1	Consolidated Functional Results	28
Table 6.2	Observed Output Characteristics	28

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CNN	Convolutional Neural Network
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CUDA	Compute Unified Device Architecture
DFD	Data Flow Diagram
DOCX	Office Open XML Document Format
ER	Entity Relationship
HNSW	Hierarchical Navigable Small World
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MCQ	Multiple Choice Question
NLP	Natural Language Processing
ORM	Object Relational Mapping
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SSE	Server-Sent Events
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

1. INTRODUCTION

1.1 Background

Examination preparation involves a large and mixed collection of study material — lecture slides, textbook chapters, laboratory manuals, previous question papers and online articles. The difficulty is not the absence of information but the cost of locating a specific piece of it. Search inside a document is lexical: the reader must already know the exact word used in the text. A student searching for “why does a network stop learning in deep layers” will not find the passage on the vanishing gradient problem, because the two phrasings share almost no words.

Large Language Models (LLMs) can answer questions in fluent natural language, but used directly they answer from their training data rather than from the student’s prescribed syllabus. The answer may not match the definition or notation used in the course, may state fabricated facts with full confidence, and quotes no source that the student can check.

Retrieval-Augmented Generation (RAG) addresses this. Instead of asking the model to recall an answer, the system first retrieves the passages of the student’s own documents that are semantically closest to the question, then asks the model to answer using only those passages. Retrieval works on embeddings — numerical vectors that place text with similar meaning close together — so a question and a passage match even when they share no vocabulary. The answer is grounded in an identifiable part of the student’s material, and the system can name the document it came from.

1.2 Problem Statement

Students cannot search their own material by meaning; keyword search fails whenever the question is worded differently from the text. General AI assistants are fluent but not grounded in the prescribed material, cannot cite a source and may fabricate content. Cloud notebook services provide grounding but require an account, continuous connectivity and the upload of personal study material to a service the student does not control. Preparing revision aids by hand — practice questions, concept maps, summary slides — is effective but too slow to repeat for every unit.

A self-hostable study workspace is therefore needed: one that indexes the student's own documents, answers questions by retrieving from them, cites the source of every answer, and derives revision aids automatically from the same index.

1.3 Objectives

1. To implement a RAG pipeline that indexes study documents and answers questions from the indexed content.
2. To ingest PDF, DOCX, TXT and Markdown files as well as web pages supplied as URLs.
3. To chunk extracted text with overlap and store the embedding vectors in a local vector database, one isolated collection per notebook.
4. To retrieve the most relevant chunks by cosine similarity and build a grounded prompt.
5. To stream the answer token by token and display the source document with every answer.
6. To persist chat history per notebook and summarise each document on upload.
7. To generate multiple-choice quizzes with answer validation and score tracking, and to extract concepts and relationships as an interactive mind map.
8. To generate a slide deck with multiple layouts and speaker notes, exportable as a single offline HTML file, while keeping all data local and the model provider configurable.

1.4 Scope of the Project

The delivered system provides:

- Notebook management with cascading deletion of documents, vectors and history.
- Document ingestion from files and URLs, indexed in the background.
- Automatic three-to-five sentence summary of each source.
- Retrieval-augmented chat with streamed answers, source citation and persistent history.
- Quiz generation with configurable question count and automatic scoring.
- Concept mind map rendered as a draggable, zoomable node graph.
- Slide deck generation with four layouts, speaker notes and offline HTML export.

The following are outside the present scope:

- Multi-user accounts and authentication; the application is a single-user local workspace.
- Optical character recognition of scanned, image-only PDF files.
- Interpretation of diagrams, charts and mathematics rendered as images.
- Cross-notebook retrieval and automated accuracy evaluation against a labelled dataset.

2. LITERATURE REVIEW

2.1 Introduction

This chapter reviews the background on which StudyLens is built: large language models and the limits of their unassisted answers, the retrieval-augmented generation architecture that addresses them, the embeddings and vector databases that make semantic retrieval practical, chunking strategy, and the tools students use today. It closes with the research gap this project addresses.

2.2 Large Language Models and Their Limitations

A large language model is a neural network trained to predict the next token in a sequence. The transformer architecture of Vaswani et al. replaced recurrence with self-attention, and scaling it produced models capable of summarisation, question answering and instruction following without task-specific training. Three limitations matter for academic use. Knowledge is frozen at the end of training, so a lecture handout written afterwards is unknown to the model. Hallucination occurs because the model optimises for fluent continuation rather than factual correctness, giving no signal that separates a correct statement from an invented one. And there is no attribution — the model cannot say which document a claim came from. Enlarging the context window does not solve this: placing a whole textbook in every prompt is expensive, bounded by the window size, and dilutes the relevant passage among irrelevant text.

2.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation, introduced by Lewis et al., combines a parametric generator with a non-parametric retriever. Relevant documents are retrieved from an external corpus at query time and generation is conditioned on them, so the knowledge source is updated by re-indexing rather than retraining. Indexing runs once per document — extract, split, embed, store; querying runs per question — embed, retrieve, prompt.

This resolves all three limitations. Knowledge is current because it is read from the index. Hallucination is reduced because the prompt restricts the model to the retrieved passages and instructs it to say when the context is insufficient. Attribution becomes possible because metadata stored with each passage identifies its source. Surveys of

RAG identify retrieval quality as the dominant factor in answer quality: a generator cannot recover from a retrieval step that returns the wrong passages.

2.4 Embeddings and Vector Databases

An embedding model maps text to a fixed-length vector so that passages with similar meaning lie close together. Reimers and Gurevych showed with Sentence-BERT that a fine-tuned siamese network produces sentence embeddings whose cosine similarity is a usable semantic similarity score. Cosine similarity is preferred over Euclidean distance for text because it measures the angle between vectors and is therefore insensitive to the magnitude differences that arise from passage length.

Comparing a query against every stored vector is linear in corpus size, so vector databases use approximate nearest-neighbour indexes. The Hierarchical Navigable Small World graph of Malkov and Yashunin gives logarithmic search with high recall. ChromaDB provides HNSW indexing, persistent storage, metadata filtering and collection isolation through a Python API without a separate server process, which suits a single-user local application.

2.5 Chunking Strategies

A document must be split before embedding, because one vector for a whole chapter averages many topics and loses specificity. Chunks that are too small lose the context that makes a passage interpretable; chunks that are too large dilute the topic signal. Overlapping windows prevent an explanation that spans a boundary from being split across two vectors. This project uses word-based chunking with 256 words and 32 words of overlap, both configurable — word-based splitting never divides a word and gives predictable chunk lengths even for PDF text with poor punctuation.

2.6 Existing Study Tools

- Manual reading with keyword search — free, but matches characters rather than meaning, explains nothing and does not scale to a semester of material.
- General assistants such as ChatGPT and Gemini — explain concepts well, but are not grounded in the prescribed material and cannot cite a source.
- Cloud notebook services such as NotebookLM — demonstrate the value of source-grounded answering, but need an account and connectivity, cannot be self-hosted

or extended, and hold the student's material on a service the student does not control.

- Manual study-aid preparation — effective but slow, and its coverage depends entirely on what the student happened to notice while reading.

Research on automatic question generation has moved from rigid template and rule-based methods to neural sequence-to-sequence methods and then to instruction-following language models, which can produce a well-formed multiple-choice item with plausible distractors directly from a passage.

2.7 Research Gap

RAG is an effective method for grounded answering, embedded vector databases make semantic retrieval practical on one machine, and instruction-following models can generate study aids from a passage. What is missing is a single system that combines all three for examination preparation while remaining under the student's control. Existing tools each solve part of the problem, and none reuses the index built for question answering to also produce quizzes, concept maps and revision decks — so a student who wants all four capabilities must use several disconnected tools and re-supply the same material to each of them.

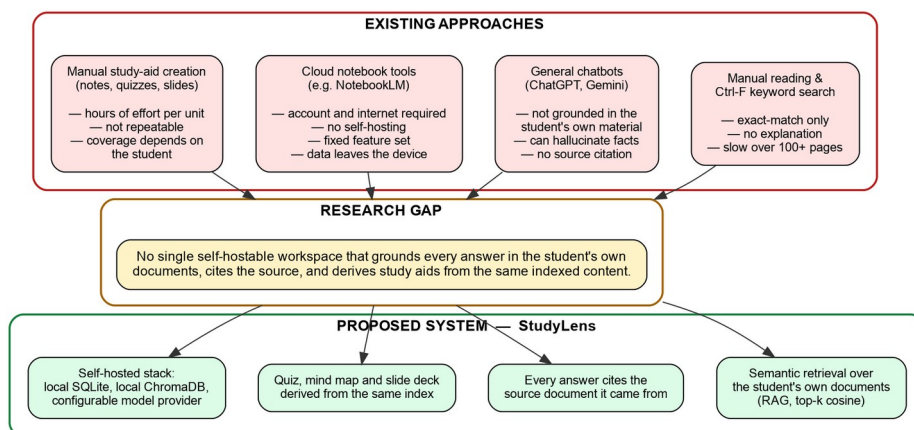


Figure 2.1: Limitations of Existing Approaches and the Research Gap

2.8 Summary

Large language models answer fluently but without grounding, currency or attribution; retrieval-augmented generation supplies all three. Chapter 3 presents the methodology that combines them.

3. METHODOLOGY

3.1 Proposed System

StudyLens is a single-user web application in which the unit of work is a notebook. A notebook represents one subject or examination unit and owns its own documents, vector collection and chat history. This isolation is a design decision: retrieval for one subject must never return a passage from another, and deleting a notebook must remove every artefact derived from it. The system is organised in three layers.

The presentation layer is a React single-page application in TypeScript, built with Vite and styled with Tailwind CSS. It provides the notebook dashboard, the source panel and four tabbed work areas — Chat, Quiz, Mind Map and Slides. It uses JSON responses for management operations and Server-Sent Event streams for all four generation features, so partial output appears as it is produced.

The application layer is a FastAPI application served by Uvicorn, exposing the REST endpoints for notebooks, documents and messages and the streaming endpoints for the four generators, and containing the ingestion, retrieval and prompt-construction logic. The data layer uses SQLite through the SQLAlchemy ORM and ChromaDB in embedded persistent mode for chunk vectors, one collection per notebook. The chat and embedding models are reached through a single provider client module — the only component that makes an outbound network call.

3.2 System Architecture

Figure 3.1 shows the architecture. Management requests are served from SQLite. Ingestion requests pass through extraction and chunking, are embedded by the provider client and written to ChromaDB. Query requests embed the question, search ChromaDB, build a grounded prompt and stream the model output back. The provider client is isolated as one module so the model backend can be replaced — for example by a locally hosted model server — without changing anything else.

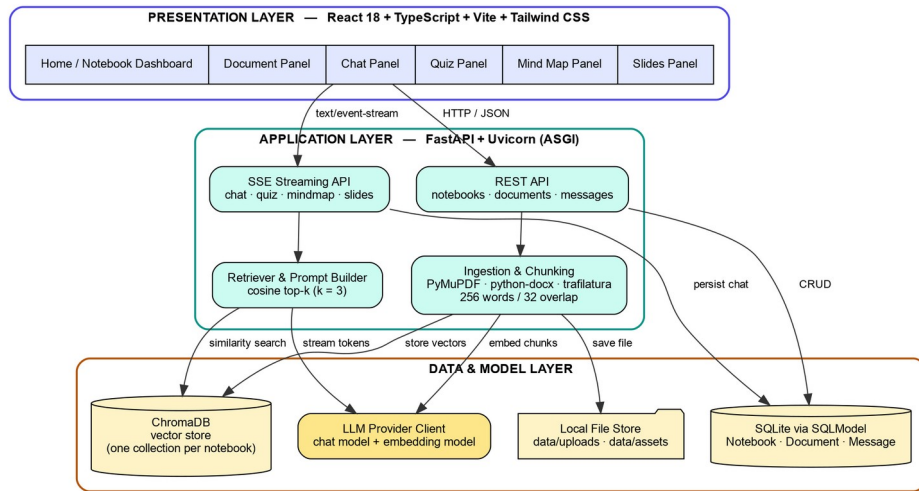


Figure 3.1: System Architecture of StudyLens

Technology	Purpose in the System
Python 3.12 / FastAPI / Uvicorn	Backend language, REST and SSE endpoints, ASGI server
SQLModel + SQLite	ORM and persistent storage of notebooks, documents, messages
ChromaDB	Embedded persistent vector store, HNSW index, cosine space
PyMuPDF / python-docx / trafilatura	Text extraction from PDF, DOCX and web pages
httpx	Asynchronous HTTP client for the model provider and URL fetching
json-repair	Recovery of malformed JSON in the slide-deck module
React 18 + TypeScript + Vite	Single-page interface, static typing, build and dev proxy
Tailwind CSS	Utility-first styling of the dark-theme interface
React Router	Client-side routing between dashboard and notebook views
@xyflow/react	Rendering of the interactive draggable mind map graph
react-markdown / lucide-react	Markdown rendering inside answers, interface icon set
torch + diffusers (optional)	Local image generation for slide visuals on a CUDA device

Table 3.1: Technologies Used in the System

3.3 Major System Modules

S. No.	Module	Responsibility
1	Notebook Manager	Create, rename, list and cascade-delete notebooks
2	Ingestion Module	Format detection, text extraction, overlapping word chunking
3	Embedding Client	Batch conversion of chunks and questions into vectors
4	Vector Store Manager	Collection creation, upsert, query and filtered delete
5	Retriever & Prompt Builder	Cosine top-k search and grounded prompt assembly
6	Streaming Gateway	Parse provider token deltas, re-emit as SSE frames
7	Summary Generator	Three-to-five sentence abstract produced at ingestion
8	Quiz / Mind Map / Slide Generators	Study-aid generation and structured parsing
9	Export Engine	Serialise a deck into one self-contained offline HTML file

S. No.	Module	Responsibility
10	History Manager	Persist and reload chat turns with their citations

Table 3.2: Major System Modules

3.4 Document Ingestion and Indexing

The upload endpoint validates the file extension, writes the file under a generated UUID name, inserts a document record with a chunk count of zero and schedules indexing as a background task before returning. The interface polls every three seconds and shows an indexing indicator until the chunk count becomes non-zero.

The background task extracts text with the parser matching the source type, splits it into overlapping chunks, requests embeddings for all chunks in one batch and upserts them into the notebook's collection with metadata holding the document identifier, document name and chunk index. The chunk count is committed at this point, which unlocks all four generation features. The summary is generated afterwards and committed separately, so a slow summary never delays usability.

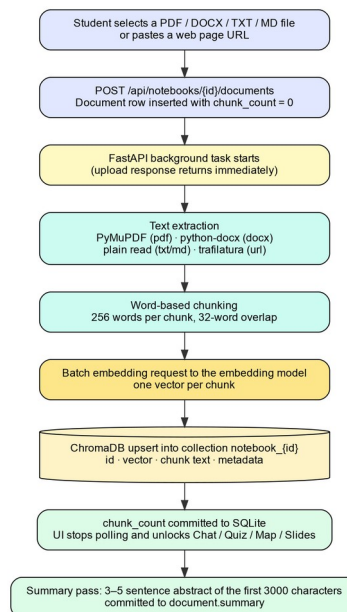


Figure 3.2: Document Ingestion and Indexing Pipeline

3.5 Retrieval-Augmented Query Workflow

A submitted question is embedded with the same model used at ingestion — a different model would place the query in a different vector space and make the distances meaningless — and the collection is queried for the three nearest chunks by cosine similarity. The retrieved chunks form the context block, the last four conversation turns

are appended so follow-up questions stay interpretable, and the system instruction directs the model to answer only from the supplied context and to say clearly when it is insufficient. That instruction is the mechanism that suppresses ungrounded answers.

The distinct source document names are emitted as the first Server-Sent Event, before generation begins, so the citation appears while the answer is still being written. Token events follow, then a completion event; the question and answer are then persisted with their source metadata.

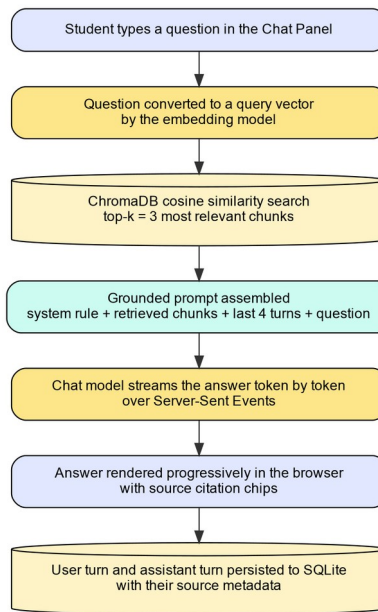


Figure 3.3: Retrieval-Augmented Query Workflow

Parameter	Value	Effect on Behaviour
CHUNK_SIZE	256 words	Larger values add context per chunk but dilute the topic signal
CHUNK_OVERLAP	32 words	Stops an explanation being lost across a chunk boundary
RAG_TOP_K	3 chunks	More chunks improve recall but lengthen the prompt
Vector space	cosine	Angle-based similarity, insensitive to passage length
History window	4 turns	Bounds prompt growth while keeping follow-ups interpretable

Table 3.3: Configurable Retrieval Parameters

3.6 Study-Aid Generation

None of the three generators re-reads the original file. Each fetches the stored chunk text for the selected document from the vector store, orders it by chunk index and reassembles it, so a document is parsed exactly once and a URL fetched exactly once.

Quiz generation sends the first 5000 characters with a prompt fixing the output format for each item — a question line, four labelled options and an answer line. Parsing splits on question markers, extracts each option with a line-anchored pattern and accepts an item only when four options and a valid answer letter are present, so a malformed item is discarded rather than shown incorrectly.

Mind map generation sends the first 2500 characters and requests a strict JSON object of nodes and labelled edges. The backend parses the outermost JSON braces and reports a parse-error flag rather than failing. The frontend lays the nodes out on a circle whose radius scales with node count and renders the graph with React Flow, so nodes can be dragged and the canvas panned and zoomed.

Slide generation sends the first 4000 characters with a fixed JSON schema: a deck title and a list of slides, each with a number, layout type, title, bullets, speaker notes and a visual prompt. The layout type is restricted to four values and the prompt requires eight to twelve slides, a title slide first and a takeaways slide last. JSON response mode is requested and the output is passed through a repair step before parsing, so a trailing comma does not discard the whole deck.

3.7 Use Case Model

The system has one human actor, the student, and one external system, the model provider. Six of the ten use cases call the provider; the other four — notebook management, history review, document deletion and offline export — are served entirely from local state and keep working without network access.

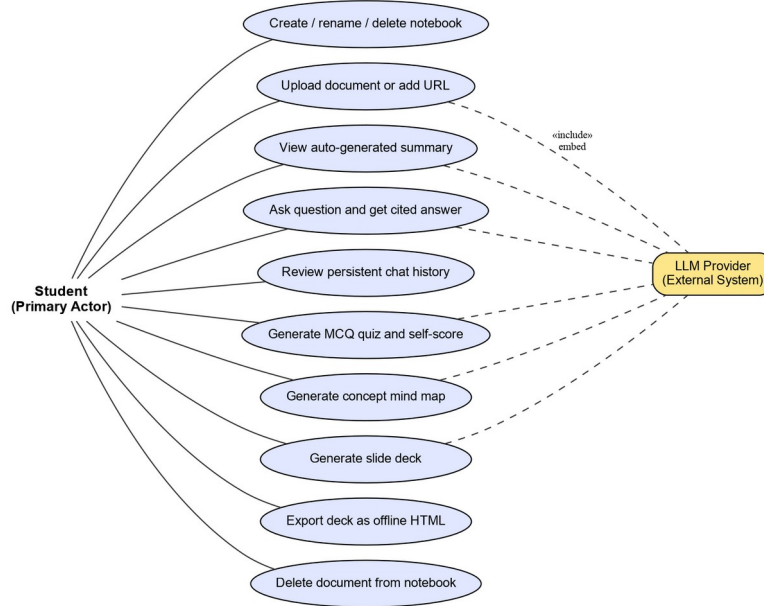


Figure 3.4: Use Case Diagram of StudyLens

3.8 Data Flow

Four processes operate over three data stores. Process 1.0 manages notebook records. Process 2.0 ingests a source, storing the file, the vectors and the chunk count and summary. Process 3.0 answers a question from the top-k chunks and writes the chat turns back. Process 4.0 generates study aids from the stored chunk text. Process 1.0 manages notebook records. Process 2.0 ingests a source, storing the file, the vectors and the chunk count and summary. Process 3.0 answers a question from the top-k chunks and writes the chat turns back. Process 4.0 generates study aids from the stored chunk text.

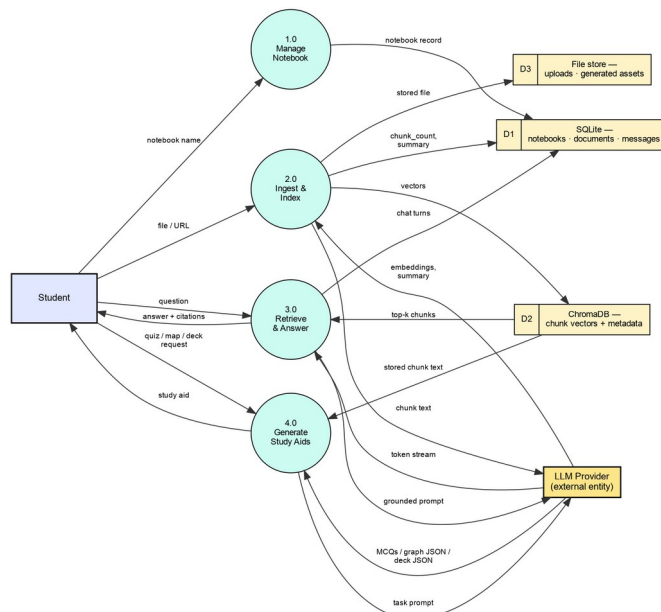


Figure 3.5: Level-1 Data Flow Diagram

3.9 Sequence of a Chat Request

Figure 3.6 shows the ordered interaction for one grounded question. Two points are worth noting: the source citation is emitted before the first answer token, and persistence happens only after streaming completes, so an interrupted generation never writes a partial answer into the history.

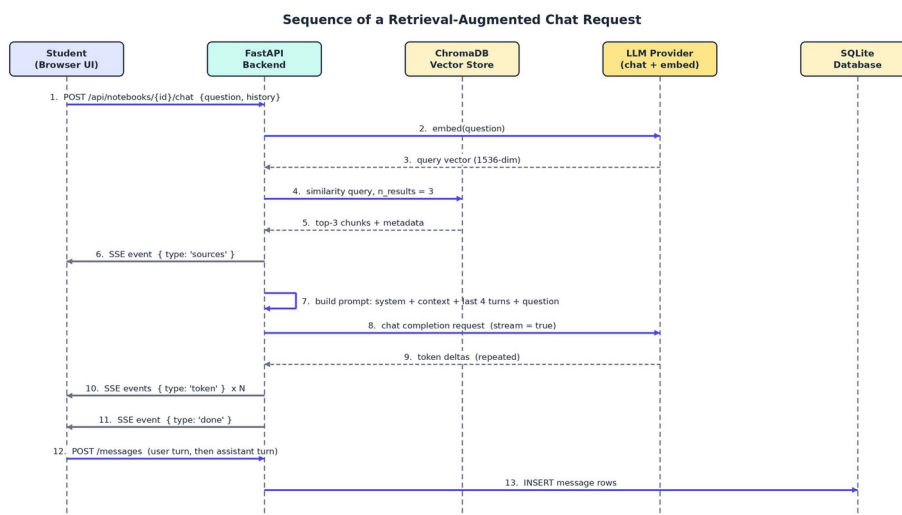


Figure 3.6: Sequence Diagram of a Chat Request

3.10 Summary

The methodology defines a three-layer system in which the notebook is the unit of isolation. Ingestion extracts, chunks and embeds a source once and stores the metadata needed for citation. Querying retrieves the three nearest chunks, constrains the model to them and streams the answer with its citation. The study-aid generators reuse the stored chunk text. Chapter 4 describes the implementation.

4. IMPLEMENTATION

4.1 Development and Testing Environment

Component	Specification
Operating system	Linux (kernel 7.0), x86-64 multi-core CPU
Graphics card	NVIDIA GeForce GTX 1650, 4 GB VRAM, CUDA available
Backend runtime	Python 3.12 in an isolated virtual environment
Frontend runtime	Node.js 18 or later with npm
Servers	Uvicorn on port 8000; Vite dev server proxying /api to it
Relational store	SQLite file at data/studylens.db
Vector store	ChromaDB persistent client at data/chroma
Models	Instruction-following chat model; embedding model returning 1536-dimensional vectors
Browser	Chromium-based browser at 1440 x 900 logical resolution

Table 4.1: Development and Testing Environment

4.2 Project Structure

The backend package is organised into seven modules:

- `config.py` — loads the environment file relative to the package so the server can start from any working directory, and exposes every tunable value with a default.
- `db.py` — `SQLModel` table classes, engine creation and the FastAPI session dependency.
- `chroma.py` — lazily creates one persistent client and returns the per-notebook collection.
- `ingest.py` — the chunker, four text extractors and a dispatcher that returns an empty string on failure.
- `openrouter.py` — the model provider client: embedding, blocking generation, JSON-mode generation and token streaming.
- `prompts.py` — all system instructions and prompt templates in one place.
- `main.py` — the FastAPI application: lifespan setup, CORS, static assets and all endpoints.

4.3 Database Implementation

Three tables are defined with `SQLModel`, which derives both the validation model and the `SQLAlchemy` table from one class. `Document` and `Message` each hold a foreign key to `Notebook`. Citations are stored on the message row as a JSON string rather than in a

separate table, because they are always read and written with the message and never queried independently; this keeps the schema to three tables without losing information.

Table	Fields
notebook	id (PK) · name · created_at
document	id (PK) · notebook_id (FK) · name · source_type (pdf/docx/txt/url) · source_ref (path or URL) · summary (nullable) · chunk_count · created_at
message	id (PK) · notebook_id (FK) · role (user/assistant) · content · sources_json (JSON array of cited documents) · created_at
ChromaDB collection	id = doc{doc_id}_chunk{i} · embedding · chunk text · metadata (doc_id, doc_name, chunk_idx)

Table 4.2: Relational Database Schema

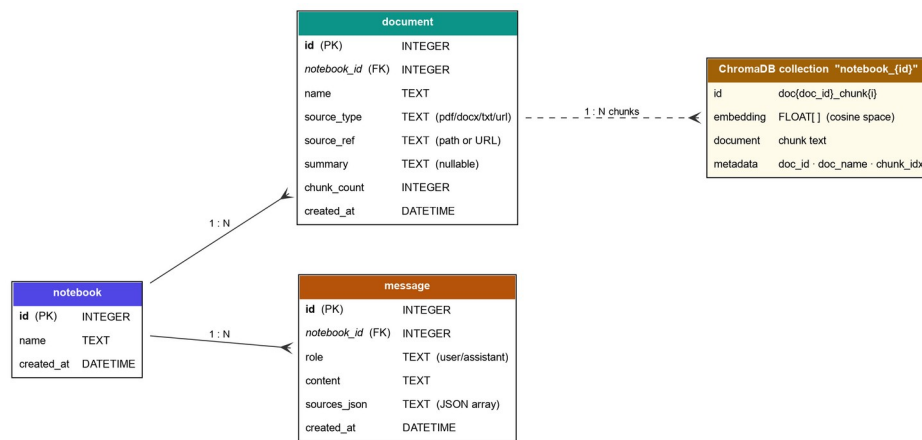


Figure 4.1: Database Schema and Vector Store Relationship

4.4 Vector Store and Ingestion

One persistent ChromaDB client is created lazily and cached at module level. Collections are named by notebook identifier and created with the cosine space set explicitly. Each chunk is stored under a deterministic identifier composed of the document identifier and the chunk index, which makes re-indexing idempotent. The stored metadata supports three operations: retrieving all chunks of one document in order for the study-aid generators, deleting one document's chunks without touching the rest of the collection, and labelling an answer with the document it came from.

The chunker splits on whitespace and advances by the chunk size minus the overlap. Four extractors are implemented: PyMuPDF for the text layer of each PDF page, python-docx for the non-empty paragraphs of a DOCX file, a direct read for plain text and Markdown, and an asynchronous fetch followed by trafilatura main-content

extraction for URLs. The dispatcher catches extraction failures and logs them, so one corrupt file cannot terminate the background task.

4.5 Model Provider Client

The provider client is the only module that performs outbound requests. It raises an explicit configuration error naming the missing variable when no API key is present. The embedding call sends all texts in one request and re-orders the returned vectors by their index field, because the response ordering is not guaranteed and a silent mis-ordering would pair every chunk with the wrong vector. Blocking generation is used for summaries, JSON-mode generation sets the structured response format, and the streaming call yields each content delta, skipping keep-alive comments and the terminating sentinel.

4.6 Streaming Endpoints and Study-Aid Modules

All four generation features are exposed as Server-Sent Event streams. The chat endpoint emits a sources frame, then one token frame per delta, then a done frame. The quiz, mind map and slide endpoints emit token frames for progress and carry the parsed result in the done frame. Each catches a read timeout from the provider and emits an error frame with a readable message, so a slow model produces an explanation rather than a stalled progress bar. Each generator reassembles the document text from the vector store and truncates it — 5000 characters for quizzes, 4000 for slides, 2500 for mind maps — bounding prompt cost according to how much context each task needs.

4.7 Offline Export and Optional Image Generation

The export function serialises the current deck into one HTML file: generated images are inlined as base64 data URIs, the deck data is embedded as a JSON literal, and the stylesheet and navigation script are written inline. The result has no external references and renders the deck with previous and next controls, slide indicators, keyboard navigation and a speaker-notes toggle, opening directly from the file system.

When image generation is enabled, the slides endpoint schedules a background task per slide using the visual prompt the model produced. The pipeline is loaded once and cached at module level, moved to the CUDA device when available, and run with four inference steps as the latent consistency model requires; generation is dispatched to a thread executor so the event loop is not blocked. The feature is disabled by default.

4.8 Frontend Implementation

The frontend is a single-page application with two routes. All network access is centralised in one API module that also declares the TypeScript interfaces for every response shape, so a change to a backend contract produces a compile-time error rather than a runtime failure. Two asynchronous generator functions handle Server-Sent Events by reading the response body stream, decoding incrementally, splitting on the frame separator and retaining any partial trailing frame in a buffer until the rest arrives.

Component	Responsibility
Home	Notebook dashboard: list, create, open and delete notebooks
Notebook	Workspace shell: header, source sidebar, tab bar, indexing poll loop
DocumentPanel	File and URL upload, source list, indexing state, summary, deletion
ChatPanel	History load, question submission, token rendering, citation chips
QuizPanel	Document and count selection, quiz parsing, validation, score display
MindMapPanel	Graph request, radial node layout, React Flow rendering
SlidesPanel	Deck request, four layout renderers, navigation, notes, export
ProgressBar	Shared token-based progress indicator for all streaming features

Table 4.3: Frontend Components

4.9 API Surface

Method	Endpoint	Purpose
GET	/api/notebooks	List notebooks, newest first
POST	/api/notebooks	Create a notebook
PATCH	/api/notebooks/{id}	Rename a notebook
DELETE	/api/notebooks/{id}	Delete a notebook, its collection, files and history
GET	/api/notebooks/{id}/documents	List documents with indexing state and summary
POST	/api/notebooks/{id}/documents	Upload a file or add a URL; schedules indexing
DELETE	/api/notebooks/{id}/documents/{doc}	Delete a document and its chunks
GET / POST / DELETE	/api/notebooks/{id}/messages	Load, persist and clear chat history
POST	/api/notebooks/{id}/chat	Grounded answer as an SSE stream
POST	/api/documents/{doc}/quiz	Multiple-choice quiz as an SSE stream
POST	/api/documents/{doc}/mindmap	Concept graph as an SSE stream
POST	/api/documents/{doc}/slides	Slide deck as an SSE stream
GET	/assets/{file}	Static delivery of generated slide images

Table 4.4: REST and Streaming API Endpoints

5. TESTING & DEPLOYMENT

5.1 Testing Objectives

Testing was carried out at three levels: automated unit tests of the model provider client against a substituted HTTP client, functional tests of every user-visible feature through the running application in a browser, and integration tests confirming that the frontend, backend, relational store, vector store and provider work together. A separate measurement run recorded the response time of every generation feature. The objectives were to verify that:

- Notebook creation and deletion behave correctly and deletion removes the associated vectors, files and history.
- PDF, DOCX, TXT, Markdown and URL sources are ingested, indexed and summarised.
- A question returns an answer grounded in the uploaded material with the correct citation, streamed progressively, and history survives a page reload.
- Quiz items are well formed and answer validation and scoring are correct.
- Mind map and slide generation produce renderable output and the exported HTML opens without a server.
- Provider and parsing failures produce a readable message instead of a silent failure.

5.2 Unit Testing

The provider client is covered by an automated test module that substitutes the HTTP client with a fake implementation, so the request that would be sent and the handling of the response can be asserted without contacting the provider. All three tests passed.

S. No.	Test Case	What It Verifies	Result
1	Embeddings returned in input order	Vectors are re-ordered by index, so chunk N is paired with vector N even when the response arrives out of order	Passed
2	JSON generation requests JSON mode	The structured response format flag is present in the request body	Passed
3	Stream ignores keep-alive frames	Keep-alive comments and the terminating sentinel are skipped and only content deltas are yielded	Passed

Table 5.1: Automated Unit Test Results

5.3 Functional Testing

Each feature was exercised through the browser against the running backend. A public encyclopaedia article on convolutional neural networks was used as the URL source and indexed into 68 chunks; a PDF file was used as the second source.

Test ID	Test Case	Expected Result	Result
TC01	Create notebook	Notebook created and opened in the workspace	Passed
TC02	Ingest URL source	Indexing indicator shown, then 68 chunks reported	Passed
TC03	Ingest PDF source	Document listed and indexed with its chunk count	Passed
TC04	Automatic summary	Three-to-five sentence summary shown on expansion	Passed
TC05	Grounded chat answer	Answer streamed and labelled with its source	Passed
TC06	History persistence	Turns and citations reloaded after a page reload	Passed
TC07	Quiz generation	Five well-formed items with four options each	Passed
TC08	Quiz validation	Wrong option red, correct option green, score 1 / 5	Passed
TC09	Mind map generation	Fifteen labelled nodes, draggable graph	Passed
TC10	Slide deck generation	Ten-slide deck with layout badges and navigation	Passed
TC11	Offline export	Single HTML file navigates without the application	Passed
TC12	Unsupported file type	Rejected with a message naming accepted formats	Passed
TC13	Chat before indexing	Explanatory error instead of an empty answer	Passed
TC14	Delete document	Row, chunks and stored file all removed	Passed

Table 5.2: Functional Test Cases

5.4 Integration Testing

- Vite development server proxying /api requests to the Uvicorn backend.
- FastAPI to SQLite through the SQLAlchemy session dependency for all CRUD operations.
- FastAPI to ChromaDB for collection creation, batch upsert, filtered retrieval, similarity query and filtered deletion.
- Backend to model provider for batch embedding, summary generation, JSON-mode generation and token streaming.
- SSE frames produced by the backend and consumed by the frontend generators, including a frame split across two network reads.
- Cascading deletion of a notebook's documents, files, vector collection and history.

5.5 Defects Identified and Resolved

Two defects were found, reproduced, corrected and re-tested. The first blocked every generation feature; the second silently corrupted displayed output.

Defect 1. Chat, quiz, mind map and slide generation all returned no output: the interface showed the source citation and a progress bar fixed at zero per cent, and the backend log recorded an exception stating that streaming response content had been accessed without the body first being read. The shared error-checking helper was parsing the response body as JSON to extract a provider error message before checking whether the response was an error at all. On a streaming response the body has not been read at that point, so the attempt raised an exception on every request, including successful ones. The helper now returns immediately when the response is not an error, and the streaming call site reads the body before parsing it when an error is present.

Defect 2. Generated quizzes displayed corrupted options — in one item, option A read “convolutional neural network (CNN)?”, a fragment of the question rather than an answer choice. The option-extraction pattern searched the whole question block for an option letter followed by a delimiter, with whitespace in the delimiter set and a case-insensitive match, so the stray article “a” inside the question sentence matched the pattern for option A. The pattern was replaced with a line-anchored, case-sensitive one requiring the letter to begin a line and be followed by a period or closing parenthesis.

S. No.	Defect	Effect	Resolution	Status
1	Error helper parsed the body of an unread streaming response	All four generation features returned no tokens	Return early when not an error; read the body before parsing on error	Fixed
2	Quiz option pattern not anchored to the start of a line	Option text captured from inside the question sentence	Line-anchored, case-sensitive pattern with a period or parenthesis delimiter	Fixed

Table 5.3: Defects Identified and Resolved

5.6 Performance Measurement

Response times were measured by a script driving the running API directly. Ingestion was timed from the upload request to the commit of the chunk count and again to the commit of the summary. Chat was measured over three questions and the mean reported, separating time to first token from time to completion.

Operation	Measured Value	Notes
Indexing a 68-chunk web source	15.25 s	Extraction, chunking, one batch embedding request, upsert
Automatic summary	4.04 s	Runs after the chunk count is committed
Chat — first token	2.42 s	Mean of three questions
Chat — complete answer	3.38 s	Mean answer length 86 streamed tokens
Quiz — five questions	4.01 s	First token after 1.03 s; 232 tokens streamed
Mind map	8.71 s	15 nodes and 13 labelled relationships
Slide deck	15.94 s	10 slides, within the 8–12 slide constraint

Table 5.4: Measured Response Times

The pattern follows from the amount of output each task produces. Chat is fastest because an answer is short; quiz generation produces more structured text; mind map generation must emit strict JSON; and slide generation produces the largest object, with bullets, notes and a visual prompt for each of ten slides. In every case the first token arrives within a few seconds, so the interface never appears frozen.

5.7 Deployment

The application runs locally in two processes. The backend uses a virtual environment with the requirements installed and the API key set in the environment file, started with Uvicorn on port 8000; on start-up it creates the storage directories and issues the table creation statements, so no manual migration is needed. The frontend runs on the Vite server, which proxies /api to the backend so the browser sees a single origin. For a static deployment the frontend is built and served by any web server with /api reverse-proxied. Because SQLite, ChromaDB and the uploads share one data directory, backup is a matter of copying that directory.

6. RESULTS AND DISCUSSION

6.1 Introduction

All screenshots were captured from the running application against the live backend, using a notebook containing two real sources: an encyclopaedia article on convolutional neural networks indexed into 68 chunks, and a PDF indexed into one chunk. No output shown here is simulated.

6.2 Notebook Dashboard and Source Ingestion

The dashboard lists the student's notebooks and provides creation and deletion controls; opening one loads only its own sources, vectors and history. The workspace places the source panel on the left and four work areas on the right, and each indexed source shows its chunk count in green, or an animated indicator while indexing is still running.

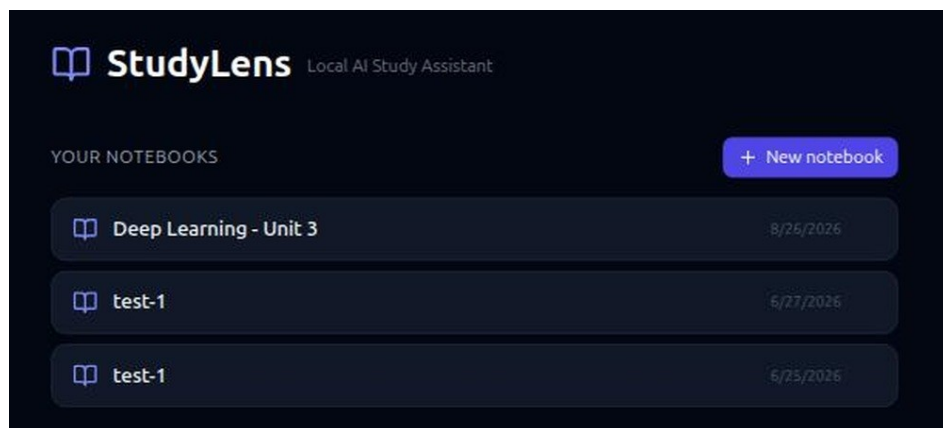


Figure 6.1: Notebook Dashboard

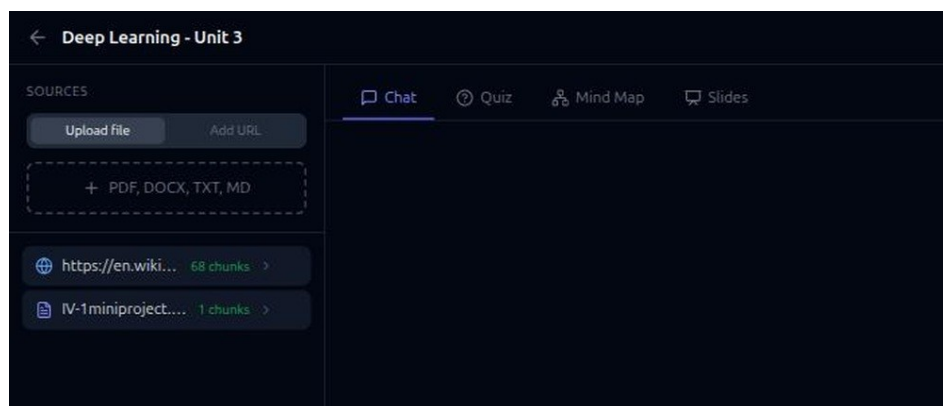


Figure 6.2: Notebook Workspace with Indexed Sources

Expanding a source reveals the summary generated automatically at ingestion. The summary in Figure 6.3 identifies the convolutional neural network as a specialised feedforward network that optimises filters, notes the role of shared weights in preventing vanishing and exploding gradients, lists the principal application areas and mentions the biological inspiration from the animal visual cortex — so it reflects the actual content of the source rather than generic text about the topic.

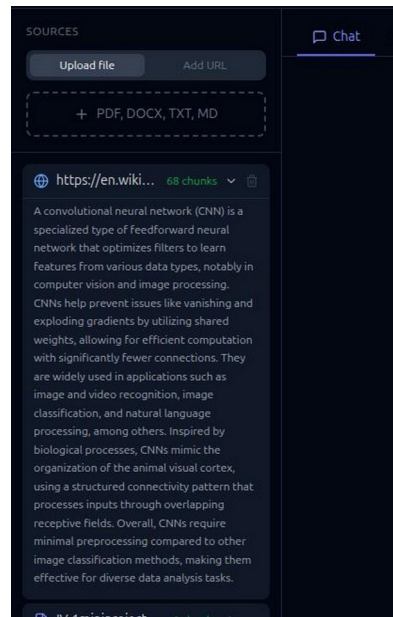


Figure 6.3: Automatically Generated Document Summary

Result: notebook management, file and URL ingestion, background indexing and automatic summarisation were verified successfully.

6.3 Retrieval-Augmented Chat

Figure 6.4 shows the state a few seconds after a question was submitted: the citation chip naming the encyclopaedia article is already displayed and the progress indicator is active, because the sources frame is emitted before generation begins.

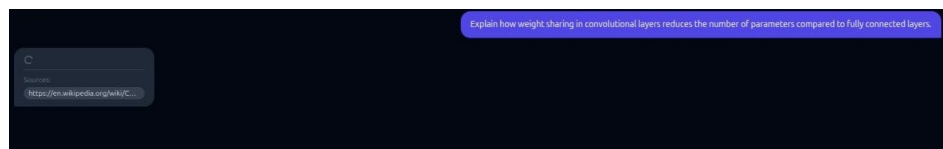


Figure 6.4: Streaming Answer with Source Citation

Figure 6.5 shows the completed answer to “Explain how weight sharing in convolutional layers reduces the number of parameters compared to fully connected layers.” The answer states that the same filter is applied across the entire input volume

so all neurons in a convolutional layer respond to the same feature within their response fields, contrasts this with fully connected layers where every connection carries its own weight, and notes that the reduction in free parameters lowers memory requirements and enables larger networks. The content is drawn from the retrieved passages and the chip names the source.

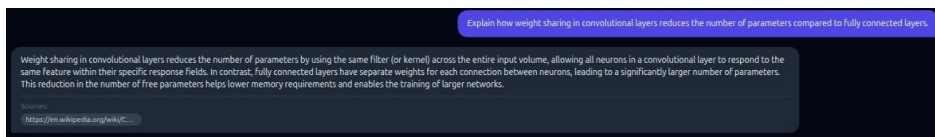


Figure 6.5: Completed Grounded Answer

Result: semantic retrieval, grounded generation, token streaming and source citation were verified successfully.

6.4 Quiz Generation and Self-Assessment

Figure 6.6 shows a five-question quiz generated from the indexed source, each item parsed into four labelled options. The questions address content specific to the source — the primary use of a convolutional neural network, the characteristic that prevents vanishing and exploding gradients, and the application areas mentioned in the text.

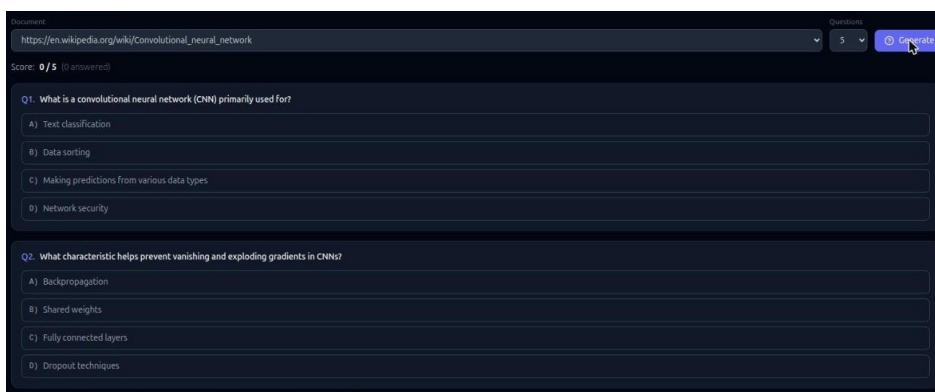


Figure 6.6: Generated Multiple-Choice Quiz

Figure 6.7 shows the state after two options were selected. In the first question an incorrect option was chosen: it is outlined in red with a cross while the correct option is simultaneously outlined in green with a tick, so the correction is immediate. The second question was answered correctly. The running score reads 1 / 5 with two questions answered, and answered items are disabled so a choice cannot be changed after the correct answer is revealed.

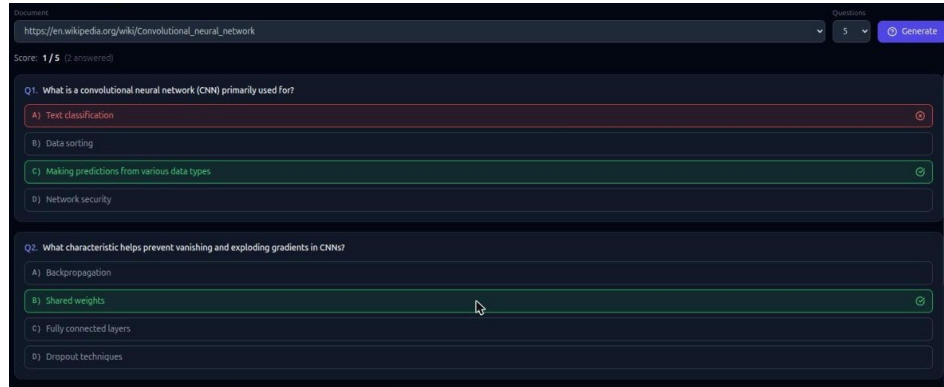


Figure 6.7: Quiz Answer Validation and Score

Result: quiz generation, option parsing, answer validation and scoring were verified successfully.

6.5 Mind Map Generation

Because the model must emit a complete JSON object, no graph can be rendered until generation finishes, so the token-based progress bar in Figure 6.8 provides the feedback that keeps the wait legible.



Figure 6.8: Mind Map Generation Progress

Figure 6.9 shows the resulting graph: fifteen concept nodes including Convolutional Neural Network, Convolution, Shared Weights, Neurons, Backpropagation, Regularization, Overfitting, Deep Learning and Transformer. The edges are labelled with the relationship they represent — “is a technique used in”, “is applied to prevent”, “comprises” — so the graph records how the concepts relate rather than merely that they co-occur.

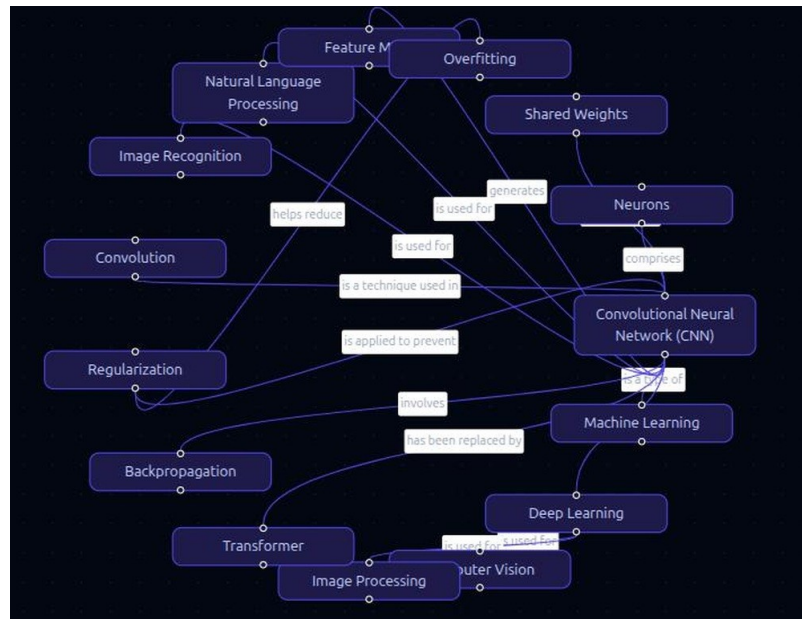


Figure 6.9: Generated Concept Mind Map

Result: concept and relationship extraction and interactive graph rendering were verified successfully.

6.6 Slide Deck Generation and Offline Export

Figure 6.10 shows the first slide of a ten-slide deck, with the layout badge reading “Title Slide” and the position indicator 1 / 10; the deck title was produced by the model from the document content. Figure 6.11 shows the second slide in the standard bullets layout, each bullet within the twelve-word limit set in the prompt.

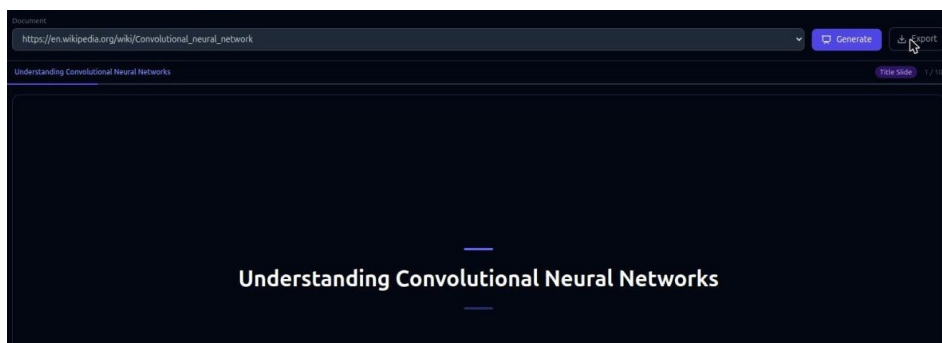


Figure 6.10: Slide Deck — Title Layout

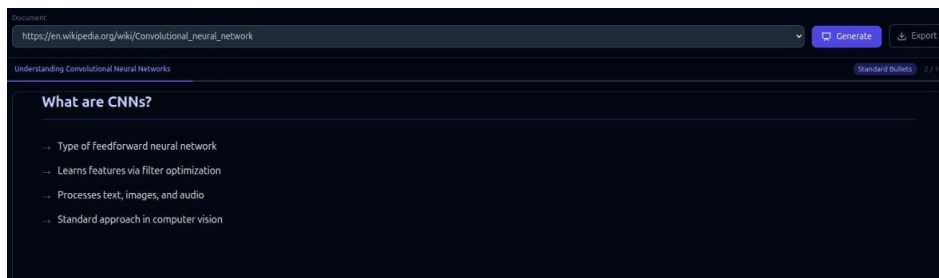


Figure 6.11: Slide Deck — Bullets Layout

Figures 6.12 and 6.13 show an exported deck opened as a standalone HTML file. The export reproduces the slides, the progress bar, the slide indicators and the navigation controls, and states the keyboard shortcuts in the header. The file contains no external references, so it renders identically without the application running and without an internet connection — the property that makes it usable for last-minute revision.

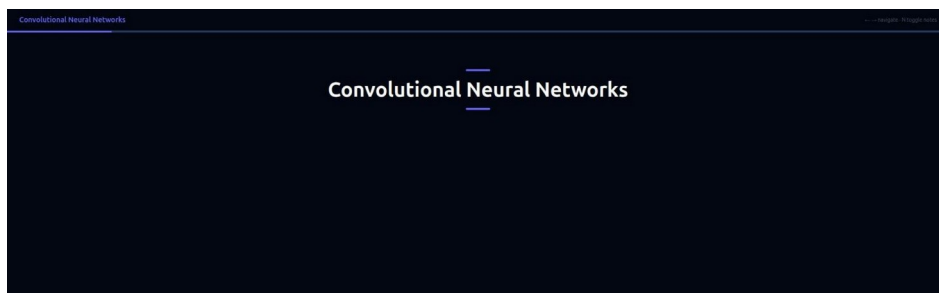


Figure 6.12: Exported Offline Deck — Title Slide



Figure 6.13: Exported Offline Deck — Content Slide

Result: structured deck generation and self-contained offline export were verified successfully.

6.7 Consolidated Results

S. No.	Functionality	Expected Result	Result
1	Notebook management	Create, list, open and delete notebooks	Passed
2	File ingestion	PDF, DOCX, TXT and Markdown indexed into chunks	Passed
3	URL ingestion	Web page content extracted and indexed	Passed
4	Background indexing	Interface responsive; state updates on completion	Passed
5	Automatic summary	Three-to-five sentence abstract per source	Passed
6	Semantic retrieval	Relevant chunks returned for a reworded question	Passed

S. No.	Functionality	Expected Result	Result
7	Grounded answering	Answer drawn from the retrieved passages	Passed
8	Answer streaming	Answer appears progressively	Passed
9	Source citation	Correct source document shown with the answer	Passed
10	History persistence	Turns and citations restored after reload	Passed
11	Quiz generation	Well-formed items with four options	Passed
12	Quiz scoring	Options marked; running score maintained	Passed
13	Mind map generation	Labelled concept graph rendered and interactive	Passed
14	Slide deck generation	Deck of 8–12 slides with layouts and notes	Passed
15	Offline export	Single HTML file navigates without a server	Passed
16	Error handling	Readable message on bad input or provider failure	Passed

Table 6.1: Consolidated Functional Results

Feature	Observed Output	Response Time
Indexing (web source)	68 chunks	15.25 s
Automatic summary	5-sentence abstract	4.04 s after indexing
Grounded chat answer	86 tokens (mean of 3)	2.42 s first token, 3.38 s total
Quiz	5 items, 4 options each	4.01 s
Mind map	15 nodes, 13 labelled edges	8.71 s
Slide deck	10 slides, 4 layout types	15.94 s

Table 6.2: Observed Output Characteristics

6.8 Discussion

The results confirm that retrieval-augmented generation solves the grounding problem identified in Chapter 1. The answer in Figure 6.5 is specific to the uploaded source and names it, so the student can verify it against the document. The question was phrased as “how does weight sharing reduce the number of parameters”, while the supporting passage discusses replicating filters across the visual field; a keyword search for the question’s wording would not have found that passage, whereas embedding-based retrieval did.

Reusing one index for four purposes was the most economical decision in the project. A document is parsed once and embedded once; the quiz, mind map and slide generators read the stored chunk text back from the vector store rather than reopening the file. The cost of each additional study aid was therefore limited to a prompt template, an endpoint and a rendering component.

Streaming affects perceived quality more than the absolute timings suggest. A ten-slide deck takes just under sixteen seconds, but the progress indicator moves within about a

second, so the wait is visibly productive. Emitting the citation before the first answer token has the same effect in the chat panel.

Testing produced two instructive findings. The first defect disabled every generation feature at once precisely because the faulty helper was shared, and a two-line correction restored all four. The second showed that output which merely looks plausible can still be wrong — the quiz appeared correct until the option text was read carefully. Limitations were also observed: retrieval depth is fixed at three chunks, the study-aid generators use a truncated prefix of the document, retrieval is purely dense so an exact equation number or rare abbreviation may be missed, and text extraction assumes an embedded text layer.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

StudyLens was developed to address a specific difficulty in examination preparation: a student holds a large volume of prescribed material but cannot search it by meaning, and the AI assistants that answer questions well are not grounded in that material and cannot cite it. The project applies the Retrieval-Augmented Generation architecture to this problem and delivers a working web application in which a student uploads their own documents and receives answers drawn from them.

The system ingests PDF, DOCX, TXT and Markdown files as well as web pages, splits their text into overlapping 256-word chunks, embeds those chunks and stores them in a local ChromaDB collection per notebook. A question is answered by embedding it with the same model, retrieving the three most similar chunks by cosine similarity, constructing a prompt that restricts the model to those chunks, and streaming the answer with the source document displayed alongside it. The same index is reused to generate a scored multiple-choice quiz, an interactive concept mind map and a slide deck that exports to a single offline HTML file. All data stays on the local machine and the model provider is reached through one isolated client module.

The system was verified against real documents: three unit tests and fourteen functional test cases passed, and all sixteen consolidated functionalities were confirmed on the running application. Measured times were 15.25 s to index a 68-chunk source, 3.38 s for a complete grounded answer, 4.01 s for a five-question quiz, 8.71 s for a fifteen-node mind map and 15.94 s for a ten-slide deck. Two defects found during testing were diagnosed, corrected and re-tested. The project therefore demonstrates that a single ingestion pass can serve grounded question answering and three distinct study-aid generators in a practical, self-hostable workspace.

7.2 Limitations

1. Retrieval depth is fixed at three chunks, which is insufficient for a question whose answer is distributed across a long chapter.
2. The study-aid generators operate on a truncated prefix of the document, so for a long source they characterise the opening sections rather than the whole.

3. Retrieval is purely dense, so an exact equation number, rare abbreviation or proper noun may be missed where a lexical method would find it.
4. Text extraction assumes an embedded text layer; scanned image-only PDF files yield no content, and diagrams and mathematics rendered as images are not interpreted.
5. Each query is answered from a single notebook, so a question spanning two subjects cannot be answered in one step.
6. The application is single-user with no authentication and is intended to run on the student's own machine.
7. Answer quality was assessed by inspection against the source documents; no automated evaluation against a labelled ground-truth dataset was performed.
8. With a hosted model provider configured, prompts and retrieved passages are transmitted to that provider; fully local operation requires a locally hosted model server.

7.3 Future Scope

1. Hybrid retrieval — fusing dense vector search with a lexical method such as BM25 to keep semantic matching while restoring the ability to find exact identifiers and rare terms.
2. Re-ranking and adaptive depth — a cross-encoder re-ranking stage over a larger candidate set, and a retrieval depth that varies with the type of question.
3. Optical character recognition and figure understanding — extending support to scanned notes, and captioning extracted figures so diagrams become retrievable.
4. Structure-aware chunking and page-level citation — splitting on headings and paragraphs rather than a fixed word count, and citing the exact page instead of the document.
5. Knowledge graph layer — extracting entities and relationships at ingestion so retrieval can traverse related concepts and support multi-hop questions.
6. Cross-notebook retrieval — an opt-in query across several notebooks for questions that span subjects, with single-notebook isolation remaining the default.
7. Spaced-repetition scheduling — recording incorrectly answered quiz items and re-presenting them on a schedule, turning the quiz into a revision programme.

8. Automated evaluation — scoring retrieval and answer quality against a known question set so chunk size, overlap and retrieval depth can be tuned against measured results.
9. Fully local model deployment — substituting a locally hosted model server for the hosted provider, which requires replacing one module rather than modifying the pipeline.
10. Additional export formats and mobile access — PDF and PowerPoint export, printable answer sheets and a responsive interface for revision away from a desktop.

REFERENCES

- [1] Lewis, P., Perez, E., Piktus, A., et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, NeurIPS, 2020.
- [2] Vaswani, A., Shazeer, N., Parmar, N., et al., “Attention Is All You Need”, NeurIPS, 2017.
- [3] Reimers, N. and Gurevych, I., “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”, EMNLP-IJCNLP, 2019.
- [4] Karpukhin, V., Oguz, B., Min, S., et al., “Dense Passage Retrieval for Open-Domain Question Answering”, EMNLP, 2020.
- [5] Gao, Y., Xiong, Y., Gao, X., et al., “Retrieval-Augmented Generation for Large Language Models: A Survey”, arXiv:2312.10997, 2023.
- [6] Malkov, Y. A. and Yashunin, D. A., “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”, IEEE TPAMI, Vol. 42, No. 4, 2020.
- [7] Robertson, S. and Zaragoza, H., “The Probabilistic Relevance Framework: BM25 and Beyond”, Foundations and Trends in Information Retrieval, Vol. 3, No. 4, 2009.
- [8] Es, S., James, J., Espinosa-Anke, L. and Schockaert, S., “RAGAS: Automated Evaluation of Retrieval Augmented Generation”, arXiv:2309.15217, 2023.
- [9] Chroma, “Chroma – the AI-native open-source embedding database”. Available at: <https://docs.trychroma.com/>
- [10] FastAPI, “FastAPI – High-performance Python Web Framework for Building APIs”. Available at: <https://fastapi.tiangolo.com/>
- [11] SQLAlchemy, “SQLAlchemy – SQL Databases in Python”. Available at: <https://sqlalchemy.org/>
- [12] React, “React – The Library for Web and Native User Interfaces”. Available at: <https://react.dev/>
- [13] Tailwind CSS, “Tailwind CSS Documentation”. Available at: <https://tailwindcss.com/docs>

- [14] React Flow, “React Flow Documentation”. Available at: <https://reactflow.dev/>
- [15] PyMuPDF, “PyMuPDF – Python Bindings for the MuPDF Library”. Available at: <https://pymupdf.readthedocs.io/>
- [16] Trafilatura, “Trafilatura – Web Scraping and Text Extraction Tool”. Available at: <https://trafilatura.readthedocs.io/>
- [17] MDN Web Docs, “Using Server-Sent Events”, Mozilla Developer Network. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- [18] Google, “NotebookLM – AI-Powered Research and Note-Taking Assistant”. Available at: <https://notebooklm.google.com/>